Databricks Data Engineering Concepts

Source: https://docs.databricks.com/aws/en/data-engineering/concepts

Extracted: August 30, 2026

Introduction

This document introduces general data engineering concepts in Databricks, covering six fundamental areas that are essential for understanding data processing and transformation on the Databricks platform.

1. Procedural vs. Declarative Data Processing

Overview

The document contrasts two fundamental programming paradigms used in data pipeline design:

Procedural approach: "specify how tasks should be accomplished by defining explicit sequences of operations"

Declarative approach: focuses on "what needs to be achieved, leaving the underlying system to determine the best way to execute the task"

Procedural Data Processing

Key Characteristics

• Step-by-step execution with developer-defined operation sequences

• Relies on control structures like loops and conditionals

• Enables fine-grained resource control and manual performance tuning

• Classified as a subset of imperative programming

Use Cases

• Custom ETL pipelines with procedural logic

• Low-level performance optimizations in batch and streaming workflows

• Legacy systems and existing imperative scripts

Implementation

Achievable through Apache Spark and Lakeflow Jobs for "step-by-step transformations and actions on distributed data"

Declarative Data Processing

Key Characteristics

• Abstracts execution details by describing desired outcomes

• Enables automatic system optimization through query planning

• Reduces complexity by eliminating explicit control structures

• Related to domain-specific and functional programming paradigms

Use Cases

• SQL-based transformations in batch and streaming workflows

• High-level data processing frameworks like Lakeflow pipelines

• Scalable, distributed workloads requiring automated optimizations

Implementation

Lakeflow pipelines use Apache Spark Declarative Pipelines (SDP), automating "orchestration, compute management, monitoring, data quality enforcement, and error handling"

Comparison Matrix

Selection Criteria

Aspect	Procedural	Declarative
Control	Full developer control	System-managed execution
Complexity	Complex and verbose	Simpler and concise
Optimization	Manual tuning required	Automatic system optimization
Flexibility	High (requires expertise)	Lower (easier to use)
Best for	Custom pipelines, performance tuning	SQL queries, managed pipelines

• Fine-grained execution control is essential

• Complex business rules resist declarative expression

• Manual performance tuning is necessary

Choose declarative processing when:

• Simplified development and maintenance matter most

• SQL-based transformations meet requirements

• Built-in framework optimizations suffice

2. Batch vs. Streaming Data Processing

Overview

"Batch and streaming are two data processing semantics used for data engineering workloads, including ingestion, transformation, and real-time processing."

Databricks treats both approaches through a unified architecture using Apache Spark and Structured Streaming, allowing sources like cloud storage and Delta Lake to function as streaming sources for incremental processing.

Batch Semantics

The batch approach processes all currently available data in a source without tracking what has already been processed. Data sources are typically partitioned (by day or region) to limit reprocessing.

Example: Calculating hourly average item sales prices reprocesses previous hours' data and overwrites earlier results with updated calculations.

Streaming Semantics

Streaming tracks processed data and handles only new information in subsequent runs. Using the same sales price example, only newly added data gets processed, and results append to previous calculations.

Late Arrival Data Challenge: If first-hour sales data arrives during the second hour, batch processing reprocesses and corrects all first-hour results, while streaming must maintain state information to properly update prior calculations.

Pros and Cons Comparison

Batch Processing

Advantages: Simple logic; results always accurate and comprehensive

Disadvantages: Inefficient due to data reprocessing; handles latency requirements from hours to minutes, not seconds

Tools: Materialized view flows, Apache Spark APIs

Streaming Processing

Advantages: Processes only new data; handles sub-second latency requirements

Disadvantages: Complex logic for stateful operations (joins, aggregations); results may not reflect all data

Tools: Lakeflow Connect, Append flows, Spark Structured Streaming APIs

Medallion Architecture Recommendations

Bronze Layer: Use streaming for ingestion workloads with stateless processing

Silver Layer: Use batch with incremental refresh; streaming only when latency outweighs accuracy concerns

Gold Layer: Use batch processing for final aggregations

3. Tables and Views

Overview

Databricks provides four core data objects for data engineering: tables, views, materialized views, and streaming tables. Each offers distinct storage and refresh characteristics.

Table

A table represents structured data stored in a specific location. "The default table type created in Databricks is a Unity Catalog managed table." These objects support standard SQL operations including INSERT, UPDATE, DELETE, and MERGE INTO commands, as well as DataFrame APIs for manipulation and querying.

View

A view functions as a virtual table constructed from a query without persisting data independently. These objects derive their content from one or more underlying tables while presenting information in a customized format. They serve to simplify intricate queries, encapsulate business requirements, and deliver standardized data access without requiring additional storage space.

Materialized View

Like views, materialized views are query-based objects. The key distinction is that they "precompute and store the query's result." This approach enables faster query performance but requires additional storage capacity. Creation and refresh can occur as standalone objects or through Lakeflow pipelines alongside other data objects.

Streaming Table

Streaming tables represent Unity Catalog managed tables incorporating processing logic via flows. These can be created independently or managed within Lakeflow pipelines, functioning alongside materialized views and views.

Materialized View vs. Streaming Table

The primary difference centers on processing semantics: materialized views employ batch operations while streaming tables use streaming operations. Selection depends on specific data engineering requirements and processing preferences.

4. Change Data Capture and Snapshots

Core Concepts

Change Data Capture (CDC) enables data engineers to replicate upstream data sources into Databricks while maintaining synchronization. Rather than processing entire datasets, CDC focuses on modifications—inserts, updates, and deletes—allowing analytical tables to stay current with operational systems.

Key Benefits

CDC provides three primary advantages:

• Reduced data volume through incremental processing rather than full dataset transfers

• Historical reconstruction capabilities for auditing and point-in-time analysis

• Stable surrogate key maintenance over time

Two Approaches to Applying Changes

SCD Type 1: Current State Only

This approach "overwrites old data with new data whenever changes occur, keeping only the latest version of each record." Best suited for scenarios requiring only present-day information and incremental materialized view refreshes.

SCD Type 2: Historical Tracking

"SCD Type 2 maintains a complete historical record by creating multiple versions of data over time, each timestamped with metadata." The `__START_AT` and `__END_AT` columns track validity periods, with active records showing `__END_AT = NULL`. Ideal for regulatory compliance, trend analysis, and understanding entity evolution.

CDC Feeds vs. Snapshots

CDC Feeds capture record-level operations with metadata including operation type and sequence numbers for deterministic ordering. Transactional databases typically generate these natively.

Snapshots represent complete table states at specific moments. Organizations use snapshots when CDC is unavailable due to costs, performance concerns, legacy system limitations, or organizational constraints.

Automated Processing Solutions

AUTO CDC API: Processes feeds from databases with CDC enabled or Delta tables with Change Data Feed enabled, automatically handling out-of-sequence records.

AUTO CDC FROM SNAPSHOT API: Compares consecutive snapshots to generate synthetic change feeds, deriving SCD logic benefits without native CDC support. This Python-only feature processes snapshots in ascending version order.

Snapshot Processing Patterns

Two methods exist for determining versions:

• **Pipeline ingestion time**: Reads snapshots at runtime using execution timestamps

• **Version functions**: Custom functions return `(DataFrame, version_number)` tuples for explicit ordering control

Additional Capabilities

• Unity Catalog `AUTO CDC` targets support INSERT, UPDATE, DELETE, and MERGE operations during pipeline execution

• Target tables emit their own Change Data Feeds for downstream consumption

• Materialized views can emit CDFs (Beta feature)

• Automatic metric capture for `num_upserted_rows` and `num_deleted_rows`

• Column-subset tracking for SCD Type 2 reduces storage while preserving critical attribute history

Implementation Recommendations

Select CDC when requiring change-only processing or historical data preservation. Choose `AUTO CDC` for native change feeds; select `AUTO CDC FROM SNAPSHOT` for periodic full dumps. Pick SCD Type 1 for current state needs or Type 2 for comprehensive historical tracking.

5. Join Operations

Overview

Databricks implements ANSI standard join syntax with support for batch and stream processing. The platform distinguishes between stateless joins (batch and stream-static) and stateful joins (stream-stream).

Join Types by Processing Mode

Batch Joins

Batch joins are stateless operations where "Results process immediately and reflect data at the time the query runs." Each execution recalculates results based on current source data. Supported join types include:

• Inner joins

• Outer joins (left, right, full)

• Semi and anti joins

• Cross joins

Databricks recommends using materialized views to optimize inner join computation.

Stream-Stream Joins

These stateful joins track information about data sources and iteratively update results. They enable powerful online processing but involve complex operational semantics. Key considerations:

• Dependent on output mode, trigger interval, and watermark configuration

• Watermarks should be specified for both sides

• Supported types: inner, left outer, right outer, full outer, and left semi joins

• Requires understanding of stateful streaming concepts

Stream-Static Joins

Stream-static joins combine a streaming data source with a static Delta Lake table using stateless operations. Characteristics include:

• Latest valid version of static data joins with current micro-batch records

• No watermarking configuration needed

• Low-latency processing capability

• Results may differ if static data changes between runs

Example Use Case: The documentation provides a Python example joining a streaming `bookings` table to a static `users` table using the `availableNow` trigger.

Join Hints

Databricks supports join hints for range joins and skew joins, with automatic optimization for skew joins. Range join hints prove useful for inequality joins on timestamps or clustering ID ranges.

6. Data Aggregation

Batch Aggregates

Batch aggregation occurs when executing ad hoc SQL queries or processing data through Apache Spark DataFrames. This approach computes aggregate statistics across all records in a data source, with Databricks optimizing performance through metadata and specialized techniques.

However, as datasets grow larger, "batch aggregation latency and compute costs can increase as data size increases." To address this challenge, Databricks recommends leveraging materialized views for pre-computing frequently accessed aggregate values.

Stateful Aggregates

In streaming workloads, aggregates maintain state and track records over time, recalculating results when new data arrives. A critical requirement exists: watermarks must be implemented in stateful aggregate queries. Without watermarks, state information accumulates infinitely, causing processing slowdowns and potential out-of-memory errors.

The documentation advises against using stateful aggregates for calculating statistics across entire datasets, recommending materialized views instead for this purpose.

Incremental Aggregates

Materialized views enable incremental computation of aggregate values. These views automatically monitor source data changes and apply corresponding updates upon refresh. Results from materialized views match those obtained from batch recomputation or ad hoc queries.

Approximate Aggregates

When precise results aren't required, approximation methods can accelerate processing and reduce costs. Spark SQL provides native functions including `approx_count_distinct`, `approx_percentile`, and `approx_top_k`. Additionally, the `TABLESAMPLE` clause generates random dataset samples for approximate aggregate calculations.

Monitoring with Aggregate Statistics

Data profiling leverages aggregate statistics and distributions to monitor data quality longitudinally, enabling trend visualization and alert scheduling for unexpected changes.

Summary

These six concepts form the foundation of data engineering on Databricks:

1. **Procedural vs. Declarative** - Understanding when to use explicit control versus declarative abstractions
2. **Batch vs. Streaming** - Choosing the right processing semantics for your latency and accuracy requirements
3. **Tables and Views** - Leveraging the appropriate data objects for storage and computation
4. **Change Data Capture** - Efficiently synchronizing data while maintaining historical context
5. **Join Operations** - Implementing joins across different processing modes
6. **Data Aggregation** - Computing aggregate statistics efficiently at scale

Mastering these concepts enables data engineers to design and implement robust, scalable data pipelines on the Databricks platform.